

Simulacija Tendermint algoritma za vizantijski tolerantan konsenzus

Seminarski rad iz predmeta Primenjeni algoritmi u upravljačkim sistemima

Student: **_Tamara Stanković_**

Broj indeksa: **_e2 6/2026_**

Dodeljeni rad: E. Buchman, J. Kwon, Z. Milošević — „The latest gossip on BFT consensus“

Novi Sad, 2026.

Sadržaj

1. Uvod i motivacija
 - 1.1. Problem konsenzusa
 - 1.2. Replikacija konačnih automata
 - 1.3. Vizantijske greške i blockchain
 - 1.4. Definicija problema
2. Tendermint algoritam
 - 2.1. Sistemski model
 - 2.2. Glasačka moć i kvorum
 - 2.3. Tipovi poruka
 - 2.4. Stanje procesa
 - 2.5. Izbor proposer
3. Glavne faze algoritma
 - 3.1. StartRound i faza predloga (propose)
 - 3.2. Faza prevote
 - 3.3. Faza precommit i zaključavanje
 - 3.4. Odlučivanje
 - 3.5. Tajmouti
 - 3.6. Mehanizam terminacije
 - 3.7. Pregled „upon“ pravila
4. Arhitektura simulatora
 - 4.1. Izbor jezika i opšti pregled
 - 4.2. Model konkurentnosti

- 4.3. Mrežni sloj i GST
- 4.4. Simulacija grešaka
- 4.5. Različita glasačka moć
- 4.6. Scenariji
- 4.7. Provera svojstava

5. Zaključak

Literatura

1. Uvod i motivacija

1.1. Problem konsenzusa

Konsenzus je jedan od najfundamentalnijih problema u distribuiranom računarstvu. Suština problema je da se grupa procesa, odnosno čvorova, koji međusobno komuniciraju isključivo razmenom poruka i od kojih neki mogu otkazati ili se ponašati zlonamerno, usaglasi oko jedne zajedničke vrednosti. Problem postaje težak u prisustvu grešaka i nepouz dane mreže, gde poruke mogu kasniti, biti izgubljene ili stizati u proizvoljnom redosledu.

1.2. Replikacija konačnih automata

Značaj konsenzusa proizilazi iz njegove uloge u replikaciji konačnih automata (engl. State Machine Replication, SMR). SMR je opšti pristup repliciranju servisa koji se mogu modelovati kao deterministički konačni automat. Ideja je da sve replike kreću iz istog početnog stanja i izvršavaju iste zahteve, odnosno transakcije, u istom redosledu, čime se garantuje da ostaju međusobno usaglašene. Uloga konsenzusa u SMR pristupu je da obezbedi da sve replike prime transakcije u istom redosledu.

1.3. Vizantijske greške i blockchain

SMR sistemi su se tradicionalno primenjivali u okviru jedne administrativne domene, sa malim brojem replika, gde se pretpostavljalo da do grešaka dolazi samo usled otkazivanja procesa (engl. crash faults). Pojavom kriptovaluta i blockchain sistema problem je dobio novu dimenziju: potrebno je postići saglasnost preko šire mreže, među velikim brojem čvorova koji ne pripadaju istoj administrativnoj domeni i od kojih se neki mogu ponašati proizvoljno, odnosno zlonamerno.

Takve greške nazivaju se vizantijskim greškama (engl. Byzantine faults), prema poznatom problemu vizantijskih generala. Algoritmi otporni na ovakve greške nazivaju se vizantijski tolerantnim algoritmima (engl. Byzantine Fault Tolerant, BFT).

1.4. Definicija problema

Tendermint rešava varijantu vizantijskog konsenzus problema motivisanu blockchain sistemima, poznatu kao konsenzus zasnovan na predikatu validnosti. Problem je definisan kroz tri svojstva:

- Saglasnost (Agreement): nijedna dva ispravna procesa ne odlučuju o različitim vrednostima.
- Terminacija (Termination): svi ispravni procesi na kraju odluče o nekoj vrednosti.
- Validnost (Validity): odlučena vrednost je validna, tj. zadovoljava unapred definisan predikat valid().

Predikat validnosti je specifičan za aplikaciju. U kontekstu blockchaina, na primer, vrednost, odnosno blok, nije validna ako ne sadrži ispravan heš prethodnog bloka.

2. Tendermint algoritam

Algoritam Tendermint, predstavljen u radu „The latest gossip on BFT consensus“ autora Buchman, Kwon i Milošević, predstavlja modernu i pojednostavljenu BFT konsenzus proceduru. Inspirisan je PBFT algoritmom i DLS algoritmom, ali je prilagođen radu nad gossip mrežom sa velikim brojem čvorova. Glavna novina algoritma je nov mehanizam terminacije koji ne zahteva razmenu dodatnih poruka.

2.1. Sistemski model

Algoritam pretpostavlja model delimične sinhronije (engl. partial synchrony). U ovom modelu postoji granica kašnjenja Δ i trenutak GST (engl. Global Stabilization Time) takav da je sva komunikacija među ispravnim procesima nakon GST pouzdana i Δ -pravovremena. Pre GST mreža može biti potpuno asinhrona — poruke mogu proizvoljno kasniti ili biti izgubljene. Vrednosti Δ i GST ne moraju biti poznate procesima; one su potrebne samo za dokaz terminacije, a ne i za bezbednost algoritma.

Svaki proces poseduje određenu glasačku moć (engl. voting power). Ukupna glasačka moć označava se sa n , a glasačka moć neispravnih procesa ograničena je parametrom f . Ključna pretpostavka algoritma je $n > 3f$, odnosno da je glasačka moć vizantijskih procesa strogo manja od trećine ukupne. Radi jednostavnosti, algoritam se često izlaže za slučaj $n = 3f + 1$.

2.2. Glasačka moć i kvorum

Iz pretpostavke $n > 3f$ proizilazi centralni pojam algoritma — kvorum veličine $2f + 1$. Svaki put kada se zahteva prijem „ $2f + 1$ poruka“ određenog tipa, misli se na poruke čiji pošiljaoci imaju zbirnu glasačku moć najmanje $2f + 1$.

Značaj ovog broja opisuje Lema 1 iz rada: svaka dva skupa procesa sa glasačkom moći najmanje $2f + 1$ imaju bar jedan ispravan proces zajednički. Pošto ispravan proces u jednoj rundi šalje najviše jedan glas za neku vrednost, dve različite vrednosti nikada ne mogu obe dobiti kvorum u istoj rundi. Na ovoj jednostavnoj činjenici počiva bezbednost celog algoritma.

2.3. Tipovi poruka

Procesi razmenjuju tri tipa poruka:

- PROPOSAL — predlog vrednosti koji šalje proposer, odnosno lider tekuće runde. Ovo je jedina poruka koja nosi celu vrednost, na primer blok transakcija.
- PREVOTE — glas u prvom krugu glasanja.
- PRECOMMIT — glas u drugom krugu glasanja.

Bitna optimizacija je da PROPOSAL nosi celu vrednost v , dok PREVOTE i PRECOMMIT nose samo kratak identifikator $\text{id}(v)$, odnosno heš. Pošto je $\text{id}(v)$ konstantne veličine, komunikacija ne raste sa veličinom bloka. Pritom važi da $\text{id}(v) = \text{id}(v')$ implicira $v = v'$.

2.4. Stanje procesa

Svaki proces održava sledeće promenljive:

- h (height) — tekuća visina, tj. instanca konsenzusa koja se rešava;
- round — redni broj tekuće runde u okviru date visine;
- step — tekuća faza unutar runde: propose, prevote ili precommit;
- $\text{decision}[]$ — niz odluka, po jedna za svaku visinu;
- lockedValue , lockedRound — vrednost za koju je proces poslednji put poslao PRECOMMIT i runda u kojoj je to učinio;
- validValue , validRound — poslednja vrednost koju je proces video kao moguću odluku i odgovarajuća runda.

Pojam zaključavanja (engl. locking) je suštinski za bezbednost. Kada proces pošalje PRECOMMIT za vrednost v , on se „zaključava“ za nju postavljanjem $\text{lockedValue} = v$ i $\text{lockedRound} = r$. Time se obavezuje da u kasnijim rundama neće olako glasati za drugu vrednost, što sprečava razilaženje ispravnih procesa.

2.5. Izbor proposerera

Svaka runda ima tačno jednog proposerera, određenog funkcijom $\text{proposer}(h, \text{round})$ koja je poznata svim procesima. Funkcija je oblika otežanog kružnog rasporeda (engl. weighted round-robin), gde se procesi biraju srazmerno glasačkoj moći.

U slučaju jednake glasačke moći, izbor se svodi na prostu rotaciju $\text{proposer}(h, \text{round}) = (h + \text{round}) \bmod n$, dok se za različite moći primenjuje puni otežani raspored, opisan u odeljku 4.5.

3. Glavne faze algoritma

Algoritam je u radu predstavljen kao skup „upon“ pravila koja se izvršavaju atomično — pravilo se okida čim dnevnik poruka procesa zadovolji odgovarajući uslov. Procesi prolaze kroz tri faze unutar svake runde:

propose, prevote i precommit. Tok jedne runde u idealnom slučaju je: predlog → prvi krug glasanja → zaključavanje i drugi krug glasanja → odluka.

3.1. StartRound i faza predloga (propose)

Svaka runda počinje funkcijom StartRound. Proces postavlja step na propose i proverava da li je on proposer tekuće runde. Ako jeste, šalje PROPOSAL: ukoliko već poseduje validValue, ponovo predlaže tu vrednost uz pripadajući validRound, a u suprotnom svežu vrednost dobijenu spoljnom funkcijom getValue(). Ako proces nije proposer, zakazuje tajmout timeoutPropose kako ne bi čekao lidera neograničeno.

3.2. Faza prevote

Kada proces primi PROPOSAL sa validRound = -1 dok je u fazi propose, on glasa u prvom krugu. Glasa PREVOTE za id(v) ako je vrednost validna i ako se nije zaključao ni za jednu vrednost (lockedRound = -1) ili se zaključao baš za tu vrednost (lockedValue = v). U suprotnom glasa PREVOTE nil. Nakon glasanja prelazi u fazu prevote.

Postoji i složeniji slučaj kada proposer predloži vrednost sa validRound ≥ 0 , što znači da je ta vrednost bila moguća odluka u nekoj ranijoj rundi. Tada proces prihvata predlog samo ako poseduje dokaz — $2f + 1$ PREVOTE poruka za tu vrednost iz runde validRound. Ovo pravilo je deo mehanizma terminacije i detaljnije je obrađeno u odeljku 3.6.

3.3. Faza precommit i zaključavanje

Kada proces, nakon glasanja u prvom krugu, prikupi $2f + 1$ PREVOTE poruka za istu validnu vrednost v, on se zaključava za v i prelazi u drugi krug glasanja. Postavlja lockedValue = v i lockedRound na tekuću rundu, šalje PRECOMMIT za id(v) i prelazi u fazu precommit. Istovremeno ažurira validValue i validRound, čime beleži da je v moguća odluka. Ovo je centralno pravilo algoritma, koje se u radu nalazi na liniji 36.

Ako proces umesto toga prikupi $2f + 1$ PREVOTE poruka za nil, on šalje PRECOMMIT nil.

3.4. Odlučivanje

Proces donosi konačnu odluku kada, u nekoj rundi r, poseduje PROPOSAL za vrednost v i prikupi $2f + 1$ PRECOMMIT poruka za id(v). Tada postavlja decision[h] = v i prelazi na sledeću visinu (h + 1), resetujući stanje. Bitno je napomenuti da odluka može nastati na osnovu PRECOMMIT poruka iz bilo koje runde date visine, ne nužno tekuće.

3.5. Tajmauti

Da bi se sprečilo zaglavljivanje, algoritam koristi tri tajmauta: timeoutPropose, timeoutPrevote i timeoutPrecommit. Svaki od njih raste sa brojem runde po formuli $\text{timeoutX}(r) = \text{initTimeoutX} + r \cdot \text{timeoutDelta}$, i resetuje se za svaku novu visinu. Rastuća priroda tajmauta je ključna za terminaciju: nakon GST, tajmout će pre ili kasnije postati dovoljno dug da svi ispravni procesi stignu da se usaglasu.

Tajmauti deluju kao izlaz iz svake faze: istek `timeoutPropose` dovodi do `PREVOTE nil`, istek `timeoutPrevote` do `PRECOMMIT nil`, a istek `timeoutPrecommit` pokreće novu rundu. Pored toga, ako proces primi $f + 1$ poruka iz runde veće od tekuće, odmah prelazi u tu rundu, jer prisustvo $f + 1$ poruka garantuje da je bar jedan ispravan proces već stigao u tu rundu.

3.6. Mehanizam terminacije

Glavni doprinos Tendermint algoritma je mehanizam terminacije koji ne zahteva dodatne poruke. Mehanizam se oslanja na promenljive `validValue` i `validRound`. Kada proces u rundi r primi validan `PROPOSAL` za v i odgovarajućih $2f + 1$ `PREVOTE` poruka, on ažurira `validValue` na v i `validRound` na r . Zahvaljujući prirodi gossip komunikacije, ako neki ispravan proces zaključa vrednost u rundi r tokom dobrog perioda, svi ispravni procesi će ažurirati `validValue` pre kraja te runde.

Posledica je da, kada ispravan proces postane proposer u nekoj kasnijoj rundi, on će predložiti vrednost koja je prihvatljiva svim ispravnim procesima, odnosno svoju `validValue` uz `validRound` kao dokaz. Time se garantuje da će, nakon GST, postojati runda sa ispravnim proposerom u kojoj svi ispravni procesi odlučuju. Za razliku od klasičnih BFT algoritama, ovaj mehanizam ne zahteva razmenu dodatnih poruka prilikom promene lidera.

3.7. Pregled „upon“ pravila

Naredna tabela daje pregled svih „upon“ pravila iz Algoritma 1, sa odgovarajućom metodom u implementaciji i brojem linije iz rada.

Linija	Metoda u kodu	Uslov → akcija
22	<code>_rule_proposal_initial</code>	<code>PROPOSAL</code> ($vr = -1$) u fazi <code>propose</code> → <code>PREVOTE</code> za <code>id(v)</code> ili <code>nil</code>
28	<code>_rule_proposal_with_vr</code>	<code>PROPOSAL</code> ($vr \geq 0$) + $2f + 1$ <code>PREVOTE</code> iz runde vr → <code>PREVOTE</code>
34	<code>_rule_prevote_timer</code>	$2f + 1$ <code>PREVOTE</code> bilo čega → zakaži <code>timeoutPrevote</code>
36	<code>_rule_lock_precommit</code>	$2f + 1$ <code>PREVOTE</code> za <code>id(v)</code> → zaključaj v + <code>PRECOMMIT</code>
44	<code>_rule_prevote_nil</code>	$2f + 1$ <code>PREVOTE nil</code> → <code>PRECOMMIT nil</code>
47	<code>_rule_precommit_timer</code>	$2f + 1$ <code>PRECOMMIT</code> bilo čega → zakaži <code>timeoutPrecommit</code>
49	<code>_rule_decide</code>	<code>PROPOSAL</code> + $2f + 1$ <code>PRECOMMIT</code> za <code>id(v)</code> → <code>ODLUKA</code>
55	<code>_rule_round_skip</code>	$f + 1$ poruka iz veće runde → preskoči u tu rundu

4. Arhitektura simulatora

4.1. Izbor jezika i opšti pregled

Simulator je implementiran u programskom jeziku Python. Cilj implementacije bio je da što vernije prati Algoritam 1 iz rada, uz ispunjenje svih zahteva specifikacije: distribuirani čvorovi, konkurentno izvršavanje, komunikacija porukama, simulacija grešaka i praćenje rada kroz logove. Kod je organizovan u nekoliko modula sa jasno razdvojenim odgovornostima.

Modul	Uloga
messages.py	Definicija tipova i strukture poruka; funkcija id(v), odnosno heš vrednosti.
node.py	Logika algoritma: stanje čvora, upon pravila, tajmauti, glasačka moć i izbor proposer. Sadrži klase Node i ByzantineNode.
network.py	Prenos poruka između čvorova; modelovanje kašnjenja, gubitaka i GST.
config.py	Parametri simulacije i unapred definisani scenariji.
logger.py	Thread-safe logovanje rada i provera svojstava u završnom rezimeu.
main.py	Pokretanje simulacije, kreiranje i orkestracija niti čvorova.

4.2. Model konkurentnosti

U skladu sa zahtevom specifikacije za konkurentno izvršavanje, svaki čvor je realizovan kao zasebna nit. Klasa Node nasleđuje threading.Thread. Svaka nit ima sopstvenu petlju izvršavanja, čime čvorovi rade nezavisno, kao stvarni procesi u distribuiranom sistemu. Algoritam namerno nije implementiran kao jedna sekvencijalna petlja koja redom obrađuje čvorove, što specifikacija izričito zabranjuje.

Komunikacija se odvija isključivo razmenom poruka. Svaki čvor poseduje ulazno sanduče (inbox), realizovano pomoću thread-safe reda queue.Queue. Mrežni sloj ubacuje poruke u sanduče, a nit čvora ih preuzima i obrađuje.

Vredi istaći odsustvo eksplicitnih brava oko stanja čvora. Ovo je moguće jer stanje svakog čvora menja isključivo njegova sopstvena nit; nijedna druga nit ne pristupa tuđem stanju. Jedini objekat koji prelazi granicu između niti je sanduče, koje je već sinhronizovano. Pošto ne postoji deljeno promenljivo stanje, ne može doći ni do trke podataka (engl. race condition), pa kod ostaje jednostavan i blizak pseudokodu. Brava se koristi jedino u modulu za logovanje, i to isključivo radi sprečavanja preplitanja ispisa, bez ikakvog uticaja na korektnost algoritma.

4.3. Mrežni sloj i GST

Mrežni sloj je takođe realizovan kao zasebna nit, odnosno dispatcher. Ona održava red događaja isporuke poređanih po vremenu, pomoću strukture hip, i isporučuje svaku poruku tek kada dođe njeno vreme. Ovakav dizajn omogućava izolovanu kontrolu kašnjenja i jednostavnu demonstraciju GST.

Pre trenutka GST, mreža uvodi velika kašnjenja, odnosno asinhroni „loš period“, dok nakon GST koristi mala kašnjenja, odnosno pouzdan „dobar period“. Mreža takođe modeluje gubitak poruka sa zadatom verovatnoćom, kao i pale čvorove koji niti šalju niti primaju poruke.

4.4. Simulacija grešaka

Simulator pokriva više tipova grešaka, u skladu sa prirodom algoritma. Pad čvora (crash) modeluje se tako što se odgovarajuća nit ne pokreće — čvor jednostavno ćuti. Vizantijsko ponašanje realizovano je posebnom klasom ByzantineNode, koja nasleđuje obični čvor i prepisuje samo metodu za slanje poruka, odnosno broadcast. Time je zlonamerni čvor iznutra identičan ispravnom i razlikuje se samo po načinu slanja.

Glavni vid napada je ekvivokacija: kao proposer, vizantijski čvor šalje dva različita predloga različitim grupama čvorova, pokušavajući da podeli njihove glasove tako da nijedna vrednost ne dobije kvorum. Implementiran je i mod u kome čvor uvek glasa nil.

Sušтина demonstracije je da, uprkos ekvivokaciji, bezbednost ostaje očuvana. Prema Lemi 1, dve različite vrednosti ne mogu obe dobiti kvorum u istoj rundi, pa se ispravni čvorovi ne mogu razići. Sistem se oporavlja u narednoj rundi sa ispravnim proposerom.

4.5. Različita glasačka moć

U opštem slučaju, čvorovi ne moraju imati jednaku glasačku moć. Simulator podržava zadavanje proizvoljne glasačke moći po čvoru kroz parametar voting_power u konfiguraciji. Ukoliko taj parametar nije zadat, svaki čvor ima moć 1 i sistem se ponaša kao klasična rotacija, čime je očuvana kompatibilnost sa osnovnim scenarijima.

Kada su glasačke moći različite, tri stvari se prilagođavaju u skladu sa radom.

Prvo, prilagođava se izbor proposerera. Proposer se bira otežanim kružnim rasporedom. Svakom čvoru se u svakoj rundi prioritet uvećava za njegovu glasačku moć; za proposerera se bira čvor sa najvećim prioritetom, kome se potom oduzima ukupna glasačka moć sistema. Na taj način, tokom niza od n rundi, svaki čvor postane proposer onoliko puta kolika mu je glasačka moć. Na primer, za moći $[3, 2, 1, 1]$, gde je ukupna glasačka moć 7, raspored proposerera kroz sedam rundi je N0, N1, N2, N0, N3, N1, N0. Čvor N0 je lider tri puta, N1 dva puta, a N2 i N3 po jednom.

Drugo, prilagođava se kvorum. Uslov „ $2f + 1$ “ više se ne odnosi na broj čvorova, već na zbir njihove glasačke moći. Kvorum je dostignut tek kada pošiljaoci odgovarajućih poruka imaju zbirnu glasačku moć najmanje $2f + 1$.

Treće, prilagođava se pretpostavka $n > 3f$. Ovde je n ukupna glasačka moć sistema, a f gornja granica glasačke moći vizantijskih čvorova. Za jednaku moć, ova pretpostavka se svodi na klasičan uslov nad brojem čvorova.

Ovaj mehanizam je direktno propisan radom, u kome se navodi da je funkcija izbora proposerera otežani kružni raspored srazmeran glasačkoj moći, kao i da se uslov prijema „ $2f + 1$ poruka“ odnosi na zbirnu glasačku moć pošiljalaca.

4.6. Scenariji

Radi demonstracije rada algoritma u različitim uslovima, pripremljeno je deset scenarija. Svi parametri, kao što su broj čvorova, tolerancija f , tajmauti, GST, broj visina i glasačka moć, lako se menjaju kroz konfiguraciju ili komandnu liniju, što omogućava jednostavnu demonstraciju tokom odbrane.

Scenario	Šta demonstrira
happy	Idealni uslovi, bez grešaka; odluka već u rundi 0.
crash-proposer	Pad proposera runde 0; oporavak u rundi 1 sa novim liderom, odnosno demonstracija živosti.
crash-follower	Pad običnog čvora; preostalih $2f + 1$ čvorova i dalje odlučuje.
byzantine	Ekvivokacija proposera, odnosno dva različita predloga; bezbednost je očuvana.
delays	Umerena kašnjenja manja od tajmauta; konsenzus se i dalje postiže.
asynchrony	Asinhroni period pre GST, zatim stabilizacija; terminacija nakon GST.
multi-height	Sekvenca od tri visine, odnosno tri bloka zaredom; niz instanci konsenzusa.
weighted	Različita glasačka moć [3, 2, 1, 1], ukupno 7, $f = 2$, kvorum 5; proposer se bira srazmerno moći pomoću weighted round-robin rasporeda.
weighted-byz	Različita glasačka moć i vizantijski čvor male moći, N_2 ; bezbednost je očuvana i sa težinama.
weighted-crash	Različita glasačka moć i pad najjačeg čvora, odnosno proposera runde 0; u logu se vidi smena proposera kroz runde.

4.7. Provera svojstava

Po završetku simulacije, modul za logovanje ispisuje rezime i automatski proverava tri svojstva algoritma na osnovu zabeleženih odluka ispravnih čvorova. Saglasnost se proverava upoređivanjem svih odluka na istoj visini, validnost proverom da nijedna odluka nije nil, a terminacija proverom da su svi ispravni čvorovi odlučili. Time simulator ne samo da izvršava algoritam, već i empirijski potvrđuje njegova teorijska svojstva u svakom scenariju.

5. Zaključak

U ovom radu predstavljen je Tendermint, vizantijski tolerantan konsenzus algoritam, kao i simulator koji ga verno implementira. Tendermint rešava problem konsenzusa u modelu delimične sinhronije, pod pretpostavkom $n > 3f$, kroz dva kruga glasanja, PREVOTE i PRECOMMIT, i mehanizam zaključavanja koji obezbeđuje bezbednost. Njegov glavni doprinos je mehanizam terminacije zasnovan na promenljivama `validValue` i `validRound`, koji ne zahteva razmenu dodatnih poruka prilikom promene lidera.

Implementirani simulator ispunjava sve zahteve specifikacije: čvorovi su realizovani kao nezavisne niti koje komuniciraju isključivo porukama, podržano je konkurentno izvršavanje bez deljenog stanja, a obuhvaćen je čitav niz scenarija — od idealnog slučaja, preko padova čvorova i mrežnih kašnjenja, do vizantijskog ponašanja, različite glasačke moći i sekvence više visina. Svaki scenario se završava automatskom proverom saglasnosti, validnosti i terminacije.

Kroz simulaciju je pokazano da algoritam zadržava bezbednost čak i u prisustvu zlonamernog čvora koji aktivno pokušava da prevari sistem, kao i da postiže terminaciju nakon stabilizacije mreže. Time je potvrđeno da Tendermint, uprkos svojoj jednostavnosti, predstavlja robusno rešenje pogodno za primenu u savremenim blockchain sistemima.

Kao unapređenje u odnosu na osnovnu specifikaciju, implementirana je podrška za različitu glasačku moć po čvoru, sa izborom proposer-a otežanim kružnim rasporedom i kvorumom zasnovanim na zbirnoj glasačkoj moći, čime simulator vernije prati opšti slučaj opisan u radu. Kao mogući pravci daljeg unapređenja izdvajaju se modelovanje gossip prosleđivanja poruka umesto direktne isporuke, kao i grafički prikaz toka konsenzusa.

Literatura

- [1] E. Buchman, J. Kwon, Z. Milošević, „The latest gossip on BFT consensus“, arXiv:1807.04938, 2018.
- [2] M. Castro, B. Liskov, „Practical Byzantine Fault Tolerance“, OSDI, 1999.
- [3] C. Dwork, N. Lynch, L. Stockmeyer, „Consensus in the Presence of Partial Synchrony“, JACM, 1988.
- [4] F. B. Schneider, „Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial“, ACM Computing Surveys, 1990.